Complete Volume 1

Volume 1 - Architecture Master Document

Section 1: Executive Architecture & Product Vision

1.1 Purpose

This document defines the target-state architecture for a multi-tenant Property Operations Platform designed to support:

- Single Family Residential (SFR)
- Multifamily
- RV Parks
- Mobile Home Parks
- Campgrounds
- Cabins
- Self Storage
- Mixed Asset Portfolios

The platform will operate as a cloud-native SaaS solution built on Azure and leverage ERPNext as the financial and workflow engine while exposing modern APIs and user experiences.

The objective is not to build another property management system.

The objective is to build a Property Operations Platform capable of replacing multiple systems:

Category	Typical Product
Property Management	Rent Manager
Small Landlord PM	Innago
RV Operations	Campspot
Accounting	QuickBooks

Category	Typical Product
Document Management	DocuSign + Dropbox
Utility Billing	Conservice-type systems
Reporting	Power BI

1.2 Business Vision

The platform will establish a single operating system for real estate operators.

Most operators currently use:

Property Management System

+

Accounting System

+

Reservation Platform

+

Utility Billing Platform

+

Document Platform

+

Reporting Platform

+

CRM

The target architecture consolidates those capabilities into one platform.

Unified Property Platform

├ Property Management

├ Reservations

├ Leasing

├ Accounting

├ Utility Billing

├ Reporting

├ Documents

├ Mobile Apps

└ AI Services

1.3 Strategic Objectives

Objective 1 - Asset Class Independence

Asset types are configuration, not code.

The system must support:

Property

  ├ Apartment

  ├ House

  ├ Condo

  ├ RV Site

  ├ Mobile Home Lot

  ├ Cabin

  ├ Storage Unit

  └ Commercial Suite

without creating separate software products.

Objective 2 - White Label SaaS

Property managers should be able to rebrand the platform.

Examples:

portal.hillcountrypm.com

rv.texasretreats.com

storage.lonestarstorage.com

The underlying platform remains identical.

Objective 3 - AI Native

Every domain model shall expose APIs suitable for AI agents.

Examples:

Leasing Agent

Reservation Agent

Collections Agent

Maintenance Agent

Revenue Manager

Owner Reporting Agent

1.4 Business Success Metrics

Year 1

Support:

100 customers

5,000 managed units/sites

Year 3

Support:

500 customers

50,000 managed units/sites

Year 5

Support:

2,000 customers

250,000 managed units/sites

Section 2: Business Architecture & Domain Model

2.1 Business Capability Map

Customer Management

Property Management

Lease Management

Reservation Management

Utility Management

Payments

Accounting

Documents

Reporting

Identity

Notifications

Each capability maps to an individual bounded context.

2.2 Domain Driven Design Approach

The platform will use Domain Driven Design (DDD).

Each domain owns:

- Data
- APIs
- Business Rules
- Events

No domain accesses another domain's database.

2.3 Core Business Domains

Customer Domain

Purpose:

Manage platform customers.

Entities:

Organization

Customer

Portfolio

Subscription

Plan

Property Domain

Purpose:

Manage physical assets.

Entities:

Property

Building

Unit

Lot

RV Site

Cabin

Storage Unit

Resident Domain

Purpose:

Manage occupants.

Entities:

Resident

Applicant

Lease

Move-In

Move-Out

Reservation Domain

Purpose:

Support transient occupancy.

Entities:

Reservation

Stay

Guest

Check-In

Check-Out

Financial Domain

Purpose:

Revenue operations.

Entities:

Invoice

Payment

Refund

Fee

Deposit

Utility Domain

Purpose:

Utility recovery.

Entities:

Meter

Reading

Utility Bill

Cost Allocation

Document Domain

Purpose:

Document lifecycle.

Entities:

Document

Template

Envelope

Signature Request

2.4 Asset Hierarchy

A universal hierarchy is required.

Portfolio

|

Property

|

Asset

Examples:

Apartment

Portfolio

Property

Building

Apartment

RV Park

Portfolio

RV Park

Section

Site

Storage Facility

Portfolio

Facility

Building

Storage Unit

2.5 Ownership Model

Separate ownership from occupancy.

Owner

Property

Unit

Occupant

This allows:

- Investor reporting
- Corporate ownership
- Syndications
- Joint ventures

Section 3: Hybrid Tenancy Architecture

3.1 Design Objective

The tenant architecture must support:

Very Small Landlords

Professional PMs

Enterprise Operators

White Label Resellers

while minimizing hosting costs.

3.2 Shared Tenancy Tier

Target Users:

1-100 units

Architecture:

Shared Services

Shared Database

Shared Storage

Isolation:

tenant_id

Every record must contain:

tenant_id UUID NOT NULL

Advantages

- Low cost
- Easier upgrades
- Better resource utilization

Disadvantages

- Regulatory limitations
- Limited customization

3.3 Dedicated Tenancy Tier

Target Users:

100-10,000 units

Architecture:

Dedicated Azure Resources

Dedicated Database

Dedicated Storage

Provisioning

Performed through Terraform.

terraform apply

-var customer=premierpm

Creates:

Database

Storage

DNS

SSL

Container Apps

Monitoring

3.4 White Label Tier

Designed for:

Property Managers

Franchise Operators

Regional Operators

Features:

Custom Logo

Custom Domain

Custom Email Templates

Custom Landing Pages

3.5 Tenant Routing

customer.domain.com

|

Azure Front Door

|

API Gateway

|

Tenant Resolver

|

Service Layer

3.6 Tenant Resolution Model

Tenant identification methods:

Priority 1:

Custom Domain

Priority 2:

Subdomain

Priority 3:

JWT Claims

3.7 Tenant Provisioning Lifecycle

Step 1

Create Organization

Step 2

Select Plan

Shared

Professional

Enterprise

White Label

Step 3

Provision Infrastructure

Step 4

Configure Identity

Step 5

Configure Branding

Step 6

Activate Tenant

Section 4: Azure Reference Architecture

4.1 Architecture Overview

Internet

|

Azure Front Door

|

API Gateway

|

Azure Container Apps

|

Microservices

|

PostgreSQL

Redis

Storage

4.2 Azure Service Selection

Azure Front Door

Responsibilities:

Global Routing

Web Application Firewall

SSL Termination

Load Balancing

Azure Container Apps

Chosen over AKS initially.

Benefits:

Lower Cost

Autoscaling

Managed Infrastructure

Fast Deployment

Reason:

The platform is expected to serve fewer than:

50,000 Units

during early phases.

Container Apps provide sufficient scale.

PostgreSQL Flexible Server

Purpose:

Transactional storage.

Contains:

Properties

Units

Leases

Reservations

Invoices

Payments

Redis

Purpose:

Caching

Session Storage

Rate Limiting

Distributed Locking

Azure Storage

Purpose:

Lease Files

Signed Contracts

Inspection Photos

Insurance Certificates

Invoices

Azure Key Vault

Stores:

API Keys

OAuth Secrets

Certificates

Connection Strings

No secrets may exist in source code.

Azure Monitor

Provides:

Metrics

Logs

Tracing

Alerts

4.3 Network Architecture

Front Door

Public API Gateway

Container Apps Environment

Private Database

Private Storage

Private Redis

Databases must never expose public endpoints.

4.4 Environment Strategy

Development

Purpose:

Feature Work

QA

Purpose:

Testing

UAT

Purpose:

Customer Validation

Production

Purpose:

Live Operations

4.5 Autoscaling Strategy

Scale Metric:

HTTP Requests

CPU

Memory

Queue Length

Container Apps shall automatically scale:

Min Replicas = 1

Max Replicas = 20

per service.

4.6 Disaster Recovery Targets

Shared Tier

RTO = 4 hours

RPO = 15 minutes

Enterprise Tier

RTO = 1 hour

RPO = 5 minutes

4.7 Cost Optimization Strategy

Phase 1:

Azure Container Apps

Phase 2:

Dedicated Enterprise Environments

Phase 3:

AKS only if utilization justifies it

This avoids premature Kubernetes complexity and keeps operating expenses low while supporting growth.

End of Volume 1 - Sections 1-4
Next sections: 5. Microservice Architecture, 6. Security Architecture, 7. Integration Architecture, 8. AI/Copilot Architecture.

Volume 1 - Architecture Master Document

Section 5: Microservice Architecture

5.1 Architectural Philosophy

The platform shall be implemented using a **modular microservice architecture** with clearly defined bounded contexts.

While ERPNext provides accounting, workflows, and financial controls, it shall not become the platform's system of record for all operations.

Instead:

ERPNext

=

Financial System of Record

Platform Services

=

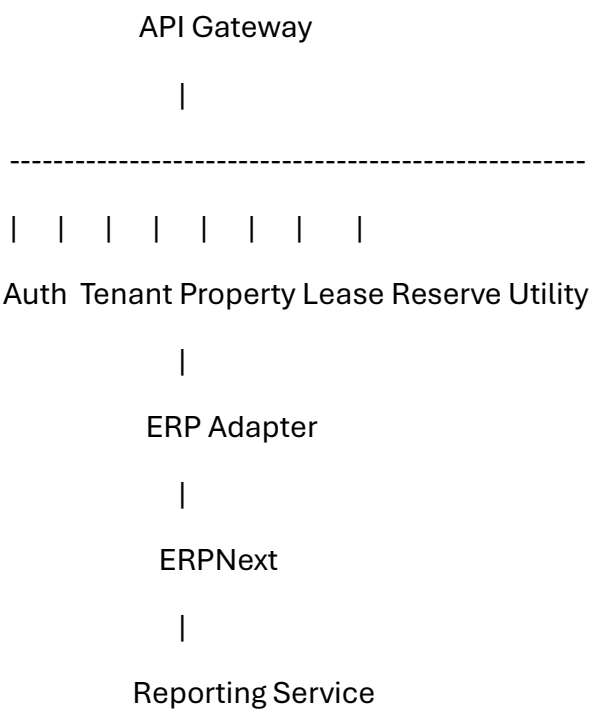
Operational System of Record

Examples:

Function	Source of Truth
Reservations	Reservation Service
Property Metadata	Property Service
Utility Readings	Utility Service

Function	Source of Truth
Maintenance Requests	Maintenance Service
General Ledger	ERPNext
AR/AP	ERPNext
Owner Statements	Reporting Service

5.2 Service Topology



All inter-service communication occurs through APIs or event streams.

5.3 Core Services

Identity Service

Responsibilities:

Authentication

Authorization

Role Management

Tenant Resolution

SSO

Owned Data:

Users

Roles

Permissions

Identity Providers

External Integrations:

Microsoft Entra

Google

Apple

Facebook

Tenant Service

Responsibilities:

Organizations

Subscription Plans

Provisioning

White Label Configuration

Owned Data:

Customers

Plans

Subscriptions

Domains

Branding

Property Service

Responsibilities:

Properties

Buildings

Units

Lots

RV Sites

Storage Units

Cabins

Owned Data:

Property Catalog

This service becomes the authoritative source for all assets.

Lease Service

Responsibilities:

Applicants

Residents

Leases

Renewals

Move-Outs

Owned Data:

Residents

Lease Contracts

Deposits

Reservation Service

Responsibilities:

Availability

Reservations

Check-In

Check-Out

Site Assignment

The Reservation Service functions similarly to Campspot.

Utility Billing Service

Responsibilities:

Meter Readings

Billing Rules

RUBS

Invoice Generation

Supported Utility Types:

Water

Electric

Sewer

Internet

Trash

Propane

Payment Service

Responsibilities:

Payment Processing

Refunds

ACH

Credit Cards

Autopay

Integrations:

Stripe

Plaid

Authorize.net

Maintenance Service

Responsibilities:

Work Orders

Inspections

Preventive Maintenance

Vendor Assignments

Future AI opportunities:

Predictive Maintenance

Vendor Recommendation

Document Service

Responsibilities:

Contracts

Leases

Photos

Insurance Records

Inspections

Integrations:

DocuSign

Adobe Sign

Dropbox Sign

Reporting Service

Responsibilities:

KPIs

Dashboards

Financial Analytics

Executive Reporting

Consumers:

Owners

Managers

Investors

AI Agents

5.4 Event Driven Communication

Events prevent service coupling.

Example:

Lease Signed

Published by:

Lease Service

Consumed by:

ERP Adapter

Document Service

Notification Service

Reporting Service

5.5 Integration Patterns

Approved Patterns:

Pattern 1

Synchronous REST

Request

Response

Use for:

Property Lookups

Authentication

Configuration

Pattern 2

Events

Publish

Subscribe

Use for:

Reservation Created

Payment Posted

Lease Executed

Pattern 3

Scheduled Synchronization

Used only for ERPNext integrations.

5.6 Service Ownership Rules

Every entity must have a single system owner.

Example:

Lease

Owner:

Lease Service

ERPNext receives a copy.

Never duplicate ownership.

Section 6 - Security Architecture

6.1 Security Philosophy

Security model:

Zero Trust

Assumption:

No user

No service

No device

No request

is trusted by default.

Every request must prove:

Identity

Authorization

Tenant Membership

Role Membership

6.2 Identity Architecture

Supported Providers:

Microsoft Entra ID

Google OAuth

Apple Sign-In

Local Account

Preferred Enterprise Provider:

Microsoft Entra

Reason:

Most PM organizations use Microsoft 365.

6.3 Authentication Flow

Client

↓

Identity Provider

↓

Identity Service

↓

JWT Token

↓

API Gateway

↓

Microservices

6.4 JWT Claims

Required Claims:

```
{
  "tenantId": "uuid",
  "userId": "uuid",
  "role": "PropertyManager",
  "subscription": "Enterprise"
}
```

6.5 Authorization Model

RBAC

Core Roles:

Platform Admin

Customer Admin

Property Manager

Owner

Resident

Guest

Vendor

Maintenance Tech

6.6 Multi-Tenant Security

Every request validates:

Tenant

User

Role

Resource Ownership

Example:

A Property Manager in:

Premier PM

cannot access:

Hill Country PM

data.

6.7 Data Protection

Encryption at Rest

AES-256

Used for:

Database

Storage

Backups

Encryption in Transit

TLS 1.3

Mandatory.

6.8 Secret Management

Secrets stored only in:

Azure Key Vault

Examples:

Stripe Keys

OAuth Secrets

Database Credentials

Signing Certificates

6.9 Security Logging

Audit every:

Login

Failed Login

Lease Signature

Payment

Admin Change

Data Export

Retention:

7 Years

6.10 Compliance Targets

Initial:

SOC2 Ready

OWASP Top 10

Future:

SOC2 Type II

PCI Compliance

Privacy Compliance

Section 7 - Integration Architecture

7.1 Integration Philosophy

The platform must be integration-friendly.

No hard-coded dependencies.

All integrations occur through:

REST APIs

Webhooks

Events

7.2 ERPNext Integration Architecture

ERPNext handles:

General Ledger

AR/AP

Journal Entries

Financial Reporting

Architecture:

Property Platform

↓

ERP Adapter

↓

ERPNext

Never allow direct calls from business services to ERPNext.

Use:

ERP Adapter Service

7.3 Payment Integrations

Preferred:

Stripe

Supports:

ACH

Cards

Autopay

Future:

Dwolla

Authorize.net

7.4 E-Sign Integrations

Primary:

DocuSign

Secondary:

Adobe Sign

7.5 Communication Integrations

Email:

SendGrid

SMS:

Twilio

Push:

Firebase

7.6 Reporting Integrations

Supported:

Power BI

Excel

CSV

Snowflake

7.7 Webhook Standards

Outbound Events:

Lease Signed

Reservation Created

Payment Posted

Invoice Generated

Work Order Closed

Webhook Delivery:

Retry

Dead Letter Queue

Monitoring

Mandatory.

7.8 Public API Strategy

Public APIs exposed through:

API Gateway

Versioning:

/api/v1

/api/v2

No breaking changes without new versions.

Section 8 - AI & Copilot Architecture

8.1 AI-First Platform Strategy

The platform shall be architected assuming AI agents become primary consumers.

Future interaction:

Human

OR

AI Agent

Both must use the same APIs.

8.2 AI Services Layer

AI Gateway

├ Leasing Agent

├ Reservation Agent

├ Utility Agent

├ Collections Agent

├ Reporting Agent

└ Maintenance Agent

8.3 Leasing Agent

Capabilities:

Answer Tenant Questions

Generate Leases

Generate Renewals

Document Summaries

Uses:

Lease Service

Document Service

ERP Adapter

8.4 Reservation Agent

Capabilities:

Find Availability

Create Reservation

Adjust Reservation

Revenue Optimization

Uses:

Reservation Service

Property Service

Pricing Engine

8.5 Reporting Agent

Capabilities:

NOI Analysis

Occupancy Analysis

Collections Analysis

Owner Summaries

Uses:

Reporting Service

ERPNext

Property Service

8.6 AI Data Architecture

AI does NOT directly access databases.

AI

↓

API Layer

↓

Services

↓

Database

This prevents unauthorized access.

8.7 Retrieval Architecture

Documents indexed into vector storage.

Sources:

Leases

Rules

Policies

Inspection Reports

Invoices

Architecture:

Document Service

↓

Indexer

↓

Vector Store

↓

AI Retrieval

8.8 AI Guardrails

Required:

Role-aware responses

Tenant-aware access

PII masking

Audit logging

Example:

An owner may ask:

Show occupancy

But not:

Show SSNs of residents

8.9 Future AI Roadmap

Phase 1

Search

Reporting

Document Summaries

Phase 2

Workflow Generation

Lease Drafting

Reservation Optimization

Phase 3

Autonomous Agents

Examples:

Collections Agent

Renewal Agent

Revenue Management Agent

Volume 1 Progress

✅ Sections 1-4 Complete
✅ Sections 5-8 Complete

Next:
Section 9 - Deployment Architecture
Section 10 - Operational Architecture
Section 11 - Non-Functional Requirements
Section 12 - Architecture Decision Records (ADRs)

These sections will complete Volume 1 and provide the foundation for Volume 2 (Data Architecture & ERD Package).

Volume 1 - Architecture Master Document

Section 9 - Deployment Architecture

9.1 Deployment Philosophy

The deployment architecture shall achieve four objectives:

1. Low initial operating cost
2. Automated customer onboarding
3. Horizontal scalability
4. Enterprise-grade availability

Infrastructure must be:

Immutable

Containerized

Terraform Managed

GitOps Driven

No production infrastructure changes shall be performed manually.

9.2 Environment Topology

The platform shall maintain four permanent environments.

Development

QA

UAT

Production

Each environment exists in a separate Azure subscription or resource group hierarchy.

Management Group

|

+-- Dev Subscription

|

+-- QA Subscription

|

+-- UAT Subscription

|

+-- Production Subscription

This design reduces blast radius.

9.3 Container Deployment Model

All services shall be packaged as OCI-compliant Docker containers.

Standard service structure:

Frontend

API Gateway

Identity Service

Tenant Service

Property Service

Lease Service

Reservation Service

Utility Service

Payment Service

Maintenance Service

Document Service

Reporting Service

ERP Adapter

Every service owns:

Container

Storage

Configuration

Health Checks

Metrics

9.4 Azure Container Apps Strategy

Container Apps shall be the preferred hosting platform through Phase 2 growth.

Reasons:

Managed Ingress

Serverless Scaling

Lower Operations Cost

Native Dapr Support

Terraform Support

Avoid AKS until:

50,000+ Units

100+ Enterprise Customers

Sustained Container Density

9.5 Shared-Tenant Deployment

Small customers use a shared architecture.

Azure Front Door

|

API Gateway

|

Shared Services

|

Shared PostgreSQL

|

Shared Storage

Isolation achieved using:

tenant_id

9.6 Enterprise Deployment

Enterprise tenants receive dedicated resources.

Customer Domain

|

Dedicated Front Door Route

|

Dedicated API Gateway

|

Dedicated Services

|

Dedicated Database

|

Dedicated Storage

Benefits:

Performance Isolation

Data Isolation

Branding

Compliance

9.7 White Label Deployment

White-label customers receive:

Dedicated Domain

Brand Assets

Custom Email Templates

Dedicated Storage

Dedicated Monitoring

Examples:

tenant.portal.com

rv.portal.com

resident.portal.com

9.8 Customer Provisioning Workflow

Provisioning is automated.

Sales

|

Subscription Activated

|

Terraform Pipeline

|

Infrastructure Created

|

Tenant Bootstrap

|

Customer Activated

Target provisioning time:

< 15 minutes

9.9 CI/CD Deployment Flow

Developer

|

Pull Request

|

GitHub Actions

|

Artifact Registry

|

Terraform Deployment

|

Container Apps

|

Smoke Tests

|

Production Release

9.10 Blue/Green Deployment Strategy

Production deployments use:

Blue Environment

Green Environment

Release Process:

Deploy

Validate

Switch Traffic

Monitor

Finalize

Rollback time:

< 5 minutes

9.11 Regional Strategy

Phase 1

US Central

Recommended:

South Central US

Phase 2

US East

US West

Phase 3

North America

Europe

Australia

9.12 Backup Architecture

Backup Categories:

Database

Object Storage

Configurations

Terraform State

Retention:

Daily 30 Days

Monthly 12 Months

Yearly 7 Years

9.13 Disaster Recovery Architecture

Primary:

South Central US

Secondary:

Central US

Failover Architecture:

Primary Region

↓

Database Replica

↓

Secondary Region

Section 10 - Operational Architecture

10.1 Operations Philosophy

Platform operations must be:

Automated

Observable

Repeatable

Auditable

Every production event must be trackable.

10.2 Site Reliability Engineering Objectives

Primary goals:

Availability

Performance

Scalability

Security

Service ownership model:

You Build It

=

You Run It

10.3 Monitoring Architecture

Sources:

Applications

Containers

Infrastructure

Databases

Integrations

Collected through:

OpenTelemetry

Exported to:

Azure Monitor

Grafana

Prometheus

10.4 Logging Architecture

All logs standardized.

Required attributes:

```
{
 "tenantId":"",
 "userId":"",
 "service":"",
 "requestId":"",
 "correlationId":""
}
```

10.5 Distributed Tracing

Required for:

Authentication

Reservations

Payments

Document Workflows

Example:

User

Gateway

Reservation Service

Payment Service

ERP Adapter

Single transaction view required.

10.6 Alerting Strategy

Severity Levels

Critical

Production Down

Database Failure

Payment Failure

Response:

Immediate Paging

High

API Performance

Integration Failures

Medium

Scaling Events

Queue Delays

Low

Informational

10.7 Support Model

Level 1

Customer Support

Level 2

Operations Team

Level 3

Engineering

10.8 Capacity Planning

Track:

Units

Customers

Transactions

Reservations

Storage

Forecast quarterly.

10.9 Business Continuity

Required capabilities:

Backup Validation

Recovery Testing

Failover Testing

Frequency:

Quarterly

10.10 Cost Monitoring

Every tenant receives:

Compute Metrics

Storage Metrics

API Metrics

Enterprise customers receive cost dashboards.

Section 11 - Non-Functional Requirements

11.1 Availability Requirements

Service Tier	Availability
Shared SaaS	99.9%
Professional	99.95%
Enterprise	99.95%+

11.2 Scalability Requirements

Platform Target:

1,000 PM Companies

250,000 Units

1,000,000 Residents

5,000,000 Documents

Without fundamental redesign.

11.3 Performance Requirements

API Response

P95 < 300 ms

Search

P95 < 2 seconds

Dashboard Load

P95 < 5 seconds

11.4 Security Requirements

Mandatory:

MFA

SSO

Audit Logging

Encryption

11.5 Privacy Requirements

Support:

Data Export

Right to Delete

Data Retention Policies

11.6 Reliability Requirements

Transaction integrity required for:

Payments

Invoices

Reservations

Lease Execution

No partial transactions.

11.7 Maintainability Requirements

Target:

Single Command Local Startup

Automated Testing

Automated Deployments

11.8 Testability Requirements

Coverage Targets:

Area	Coverage
Domain Logic	80%
APIs	80%
Critical Workflows	90%

11.9 Auditability Requirements

Track:

Lease Changes

Document Access

Payments

Administrative Changes

Retention:

7 years

11.10 AI Readiness Requirements

Every domain API must support:

Machine Consumption

Structured Metadata

Context Retrieval

Future agents shall not require schema redesign.

Section 12 - Architecture Decision Records (ADR)

ADR-001

Decision

Cloud Platform:

Microsoft Azure

Status

Approved

Rationale

Strong Microsoft ecosystem alignment.

ADR-002

Decision

ERP Platform:

ERPNext

Status

Approved

Rationale

Open-source, extensible, accounting included.

ADR-003

Decision

Frontend Framework:

Next.js

Status

Approved

Rationale

Modern UI, SSR support, strong TypeScript ecosystem.

ADR-004

Decision

Primary Database:

PostgreSQL

Status

Approved

Rationale

Scalability, partitioning, mature ecosystem.

ADR-005

Decision

Cache Layer:

Redis

Status

Approved

ADR-006

Decision

Authentication Providers:

Microsoft

Google

Apple

Status

Approved

ADR-007

Decision

Container Platform:

Azure Container Apps

Status

Approved

Reason:

Reduced operational burden versus Kubernetes.

ADR-008

Decision

Tenant Model:

Hybrid Multi-Tenant

Status

Approved

Provides:

Lowest Cost

Enterprise Isolation

White Label Flexibility

ADR-009

Decision

Infrastructure Management:

Terraform

Status

Approved

No portal-created infrastructure permitted.

ADR-010

Decision

Source Control:

GitHub

Status

Approved

Includes:

GitHub Actions

Dependabot

CodeQL

Copilot

ADR-011

Decision

API Architecture:

REST First

Event Driven Second

Status

Approved

ADR-012

Decision

AI Strategy:

AI Native Platform

Status

Approved

Every domain must expose APIs that support AI agents.

Volume 1 Complete

Volume 1 now contains:

- ✅ Executive Architecture & Vision
- ✅ Business Architecture & Domain Model
- ✅ Hybrid Tenancy Model
- ✅ Azure Reference Architecture
- ✅ Microservice Architecture
- ✅ Security Architecture
- ✅ Integration Architecture
- ✅ AI & Copilot Architecture
- ✅ Deployment Architecture
- ✅ Operational Architecture
- ✅ Non-Functional Requirements
- ✅ Architecture Decision Records

Next deliverable: Volume 2 - Data Architecture & ERD Package, which will define the canonical domain model, ERDs, PostgreSQL schemas, tenant isolation implementation, reservation model, utility billing model, and reporting data mart foundation.

Volume 2 - Data Architecture & ERD Package

Unified Property Management SaaS Platform

Canonical Domain Model, Data Architecture & Multi-Tenant Design

---

1. Purpose

Volume 2 defines the platform's canonical data architecture.

This document establishes:

- Canonical domain model
- Tenant isolation model
- PostgreSQL schema strategy
- Core entity definitions
- Reservation model
- Lease model
- Utility billing model
- Payment model
- Reporting data mart
- ERPNext integration boundaries

This document becomes the source of truth for:

- Volume 3 (OpenAPI)
- Volume 4 (Terraform)
- Volume 5 (GitHub Blueprint)
- Volume 6 (MVP Delivery Plan)

---

2. Database Strategy

2.1 Polyglot Data Architecture

The platform uses multiple data stores.

Purpose	Technology
Transaction Processing	PostgreSQL
ERP	MariaDB (ERPNext)
Cache	Redis
Document Metadata	PostgreSQL
Files	Azure Blob Storage
AI Retrieval	Vector Database
Reporting	PostgreSQL Data Mart

2.2 Source of Truth

Domain	Owner
Properties	Property Service
Leases	Lease Service
Reservations	Reservation Service
Meter Readings	Utility Service
Payments	Payment Service
Documents	Document Service
Accounting	ERPNext

---

3. Multi-Tenant Data Architecture

Shared Tier

All shared-tier customers reside in a common database.

tenant_id UUID NOT NULL

appears on every business table.

Example:

tenant_id

property_id

lease_id

reservation_id

invoice_id

payment_id

Dedicated Tier

Enterprise customers receive:

Dedicated Database

Dedicated Schema

Dedicated Storage

but retain the same logical data model.

---

4. Canonical Entity Hierarchy

Tenant

|

Portfolio

|

Property

|

Asset

|

Occupancy Model

Examples

Multifamily

Portfolio

|

Property

|

Building

|

Unit

RV Park

Portfolio

|

RV Park

|

Section

|

Site

Storage

Portfolio

|

Facility

|

Building

|

Storage Unit

---

5. Core ERD - Tenant Model

Tenant

|

+------------------+

| |

Subscription Branding

|

Organization

|

Portfolio

Tenant

tenant

---

tenant_id UUID PK

name

tenant_type

status

subscription_tier

created_at

updated_at

Organization

organization

---

organization_id UUID PK

tenant_id FK

name

tax_id

status

---

6. Property Domain Model

ERD

Property

|

+---- Building

| |

| +---- Unit

|

+---- RV Site

|

+---- Cabin

|

+---- Storage Unit

Property Table

property

---

property_id UUID PK

tenant_id UUID

property_type

name

address

status

market

Asset Table

asset

------

asset_id UUID PK

tenant_id UUID

property_id UUID

asset_type

unit_number

site_number

sqft

status

---

7. Resident & Leasing Domain

ERD

Applicant

|

Resident

|

Lease

|

Lease Amendment

Applicant

applicant

---

applicant_id UUID PK

tenant_id UUID

first_name

last_name

email

phone

status

Resident

resident

---

resident_id UUID PK

tenant_id UUID

person_type

move_in_date

move_out_date

Lease

lease

---

lease_id UUID PK

tenant_id UUID

resident_id FK

asset_id FK

start_date

end_date

monthly_rent

deposit_amount

status

Lease Status

Draft

Pending Signature

Active

Expired

Renewed

Terminated

---

8. Reservation Domain Model

Supports:

Nightly

Weekly

Monthly

Seasonal

Reservation ERD

Guest

|

Reservation

|

Assigned Site

|

Stay

Guest Table

guest

---

guest_id UUID PK

tenant_id UUID

first_name

last_name

email

Reservation Table

reservation

---

reservation_id UUID PK

tenant_id UUID

guest_id FK

asset_id FK

arrival_date

departure_date

status

total_amount

Reservation States

Draft

Reserved

Checked In

Checked Out

Cancelled

Closed

---

9. Pricing Model

Dynamic Pricing

pricing_rule

---

pricing_rule_id UUID PK

tenant_id UUID

property_id UUID

season

occupancy_threshold

multiplier

Rate Plan

rate_plan

---

rate_plan_id UUID PK

tenant_id UUID

property_id UUID

name

daily_rate

weekly_rate

monthly_rate

---

10. Utility Billing Domain

Utility ERD

Property

|

Meter

|

Reading

|

Invoice

Meter

meter

---

meter_id UUID PK

tenant_id UUID

asset_id UUID

utility_type

meter_number

Reading

meter_reading

---

reading_id UUID PK

tenant_id UUID

meter_id FK

reading_date

reading_value

Utility Invoice

utility_invoice

---

invoice_id UUID PK

tenant_id UUID

resident_id FK

billing_period

amount

---

11. RUBS Model

Support:

Water

Sewer

Trash

Methods:

Occupants

Square Footage

Unit Count

Custom Formula

Allocation Rule

allocation_rule

---

allocation_rule_id

tenant_id

method

formula

effective_date

---

12. Payments Domain

ERD

Invoice

|

Payment

|

Refund

Invoice

invoice

---

invoice_id UUID PK

tenant_id UUID

customer_id

invoice_type

invoice_date

total_amount

Payment

payment

---

payment_id UUID PK

tenant_id UUID

invoice_id FK

payment_method

amount

Supported Methods

ACH

Credit Card

Debit Card

Cash

Check

---

13. Maintenance Domain

ERD

Asset

|

Work Order

|

Vendor

Work Order

work_order

---

work_order_id UUID PK

tenant_id UUID

asset_id FK

priority

status

description

---

14. Document Domain

ERD

Document

|

Envelope

|

Signature Request

Document

document

---

document_id UUID PK

tenant_id UUID

document_type

blob_uri

status

Document Types

Lease

Addendum

Photo

Inspection

Insurance

Invoice

---

15. Notification Domain

notification

---

notification_id UUID

tenant_id UUID

recipient_id

channel

subject

status

Channels

Email

SMS

Push

In-App

---

16. Reporting Data Mart

Operational systems should not power analytics.

Reporting uses a separate model.

Star Schema

Fact Lease

Fact Reservation

Fact Revenue

Fact Payments

Fact Utilities

Fact Maintenance

Dimensions:

Date

Property

Asset

Resident

Guest

Tenant

Fact Reservation

fact_reservation

---

reservation_key

property_key

date_key

revenue

stay_nights

Fact Revenue

fact_revenue

---

invoice_key

property_key

date_key

amount

---

17. AI Data Architecture

AI Document Sources

Leases

Policy Documents

Reservations

Work Orders

Invoices

Embedding Pipeline

Document Service

|

Chunking

|

Embedding

|

Vector Store

|

AI Retrieval

---

18. ERPNext Integration Model

ERPNext receives:

Invoices

Payments

Journal Entries

Residents

Vendors

ERPNext does NOT own:

Reservations

Properties

Meters

Documents

Maintenance

---

19. Database Partitioning Strategy

Large tables partitioned by:

tenant_id

date

Tables:

reservation

invoice

payment

meter_reading

notification

---

20. Naming Standards

Tables

snake_case

Example:

meter_reading

Primary Keys

entity_id UUID

Example:

lease_id

Foreign Keys

parent_id

Example:

tenant_id

---

21. Volume 2 Deliverables

This volume establishes:

- ✅ Canonical Domain Model
- ✅ Tenant Data Strategy
- ✅ Core ERDs
- ✅ Property Model
- ✅ Lease Model
- ✅ Reservation Model
- ✅ Utility Billing Model
- ✅ Payment Model

✅ Document Model
✅ Reporting Data Mart
✅ AI Data Architecture
✅ ERPNext Ownership Boundaries
✅ PostgreSQL Standards

Volume 3 - OpenAPI Specification

Unified Property Management SaaS Platform

API Architecture, Service Contracts & Integration Standards

---

1. Purpose

This volume establishes the platform API standards and service contracts.

Objectives:

- **Standardize all APIs**
- **Enable web, mobile, AI, and partner integrations**
- **Support multi-tenancy**
- **Allow future AI agents to consume APIs**
- **Enable white-label deployments**

This specification follows:

OpenAPI: 3.1

JSON Schema: 2020-12

Authentication: OAuth2 + OIDC

Transport: HTTPS/TLS 1.3

Format: JSON

---

2. API Governance Standards

2.1 Naming Standards

Resource-oriented APIs.

Good:

GET /api/v1/properties

GET /api/v1/leases

GET /api/v1/reservations

Bad:

GET /getProperties

POST /doLease

2.2 API Versioning

Version embedded in URL.

/api/v1

/api/v2

Breaking changes require:

New Major Version

2.3 Content Types

Supported:

application/json

application/pdf

multipart/form-data

2.4 Correlation IDs

All requests require:

X-Correlation-Id

Example:

X-Correlation-Id: 74d18ffc...

---

3. Authentication APIs

Purpose

Provide:

Login

Token Validation

Provider Federation

Refresh Tokens

Logout

Endpoint

Login

POST /api/v1/auth/login

Request

```
{
 "email": "user@company.com",
 "password": "********"
}
```

Response

```
{
 "accessToken": "...",
 "refreshToken": "...",
 "expiresIn": 3600
}
```

Microsoft Login

POST /api/v1/auth/microsoft

Google Login

POST /api/v1/auth/google

Apple Login

POST /api/v1/auth/apple

Refresh Token

POST /api/v1/auth/refresh

---

4. Tenant APIs

Tenant Lookup

GET /api/v1/tenants/{id}

Create Tenant

POST /api/v1/tenants

Request

```
{
  "name": "Premier PM",
  "subscriptionTier": "Enterprise",
  "tenantType": "PropertyManager"
}
```

Update Branding

PUT /api/v1/tenants/{id}/branding

---

5. Property APIs

Property Entity

Supports:

Apartment

House

RV Park

Mobile Home

Storage

Cabin

List Properties

GET /api/v1/properties

Query Parameters

?page=1

&pageSize=50

&propertyType=RV

Create Property

POST /api/v1/properties

Request

```
{
  "name": "Texas RV Resort",
  "propertyType": "RV",
  "address": {}
```

}

Retrieve Property

GET /api/v1/properties/{propertyId}

Update Property

PUT /api/v1/properties/{propertyId}

Delete Property

DELETE /api/v1/properties/{propertyId}

---

6. Asset APIs

Assets represent:

Units

Sites

Cabins

Storage Units

Create Asset

POST /api/v1/assets

Request

{

"propertyId":"uuid",

"assetType":"RV_SITE",

"siteNumber":"A17"

}

List Assets

GET /api/v1/assets

Availability Search

GET /api/v1/assets/availability

Query

arrivalDate

departureDate

assetType

---

7. Applicant APIs

Create Applicant

POST /api/v1/applicants

Request

{

"firstName":"John",

"lastName":"Smith",

"email":"john@email.com"

}

Screening Status

GET /api/v1/applicants/{id}/screening

Approve Applicant

POST /api/v1/applicants/{id}/approve

---

8. Lease APIs

Create Lease

POST /api/v1/leases

Request

```
{
 "residentId":"uuid",
 "assetId":"uuid",
 "startDate":"2027-01-01",
 "endDate":"2027-12-31",
 "monthlyRent":1200
}
```

Get Lease

GET /api/v1/leases/{leaseId}

Generate Renewal

POST /api/v1/leases/{leaseId}/renewal

Terminate Lease

POST /api/v1/leases/{leaseId}/terminate

---

9. Reservation APIs

Search Availability

GET /api/v1/reservations/search

Request

arrivalDate

departureDate

propertyId

Response

{

"availableSites":[]

}

Create Reservation

POST /api/v1/reservations

Request

{

"guestId":"uuid",

"assetId":"uuid",

"arrivalDate":"2027-06-01",

"departureDate":"2027-06-10"

}

Check-In

POST /api/v1/reservations/{id}/checkin

Check-Out

POST /api/v1/reservations/{id}/checkout

Cancel Reservation

POST /api/v1/reservations/{id}/cancel

---

10. Utility Billing APIs

Create Meter

POST /api/v1/meters

Submit Reading

POST /api/v1/meter-readings

Request

```
{
 "meterId":"uuid",
 "readingValue":8372.44
}
```

Generate Utility Invoice

POST /api/v1/utilities/invoices/generate

Allocation Rules

POST /api/v1/utilities/allocation-rules

---

11. Invoice APIs

Generate Invoice

POST /api/v1/invoices

Retrieve Invoice

GET /api/v1/invoices/{invoiceId}

List Invoices

GET /api/v1/invoices

---

12. Payment APIs

Initiate Payment

POST /api/v1/payments

Request

```
{
 "invoiceId":"uuid",
 "amount":1200.00,
 "method":"ACH"
}
```

Refund Payment

POST /api/v1/payments/{id}/refund

Payment History

GET /api/v1/payments

---

13. Maintenance APIs

Create Work Order

POST /api/v1/work-orders

Assign Vendor

POST /api/v1/work-orders/{id}/assign

Close Work Order

POST /api/v1/work-orders/{id}/close

---

14. Document APIs

Upload Document

POST /api/v1/documents

Multipart upload.

Retrieve Document

GET /api/v1/documents/{documentId}

Request Signature

POST /api/v1/signatures/request

---

15. Notification APIs

Send Notification

POST /api/v1/notifications

Channels:

EMAIL

SMS

PUSH

IN_APP

---

16. Reporting APIs

Occupancy Dashboard

GET /api/v1/reports/occupancy

Revenue Dashboard

GET /api/v1/reports/revenue

NOI Dashboard

GET /api/v1/reports/noi

Delinquency Dashboard

GET /api/v1/reports/delinquency

---

17. Mobile APIs

Resident Mobile App:

/api/v1/mobile/resident/*

Supported:

Rent Payments

Documents

Messages

Maintenance

Guest Mobile App:

/api/v1/mobile/guest/*

Supported:

Reservations

Site Maps

Payments

Check-In

Manager Mobile App:

/api/v1/mobile/manager/*

Supported:

Inspections

Approvals

Work Orders

Reporting

---

18. Webhook Specification

Outbound events:

LeaseCreated

LeaseSigned

ReservationCreated

ReservationCancelled

InvoiceGenerated

PaymentReceived

WorkOrderCreated

WorkOrderClosed

Webhook payload:

```
{
 "eventId":"uuid",
 "eventType":"PaymentReceived",
 "occurredAt":"timestamp",
 "payload":{}
}
```

---

19. Standard Error Model

All APIs return:

```
{
  "timestamp":"2027-01-01T12:00:00Z",
  "correlationId":"uuid",
  "errorCode":"LEASE_NOT_FOUND",
  "message":"Lease could not be found"
}
```

---

20. HTTP Status Usage

200 Success

201 Created

204 No Content

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

422 Validation Error

500 Internal Error

---

21. AI Agent APIs

Future AI services interact exclusively through APIs.

Examples:

POST /api/v1/ai/leasing-agent

POST /api/v1/ai/reservation-agent

POST /api/v1/ai/reporting-agent

---

22. OpenAPI Standards

All services shall publish:

openapi: 3.1.0

Required:

Schemas

Examples

Security Definitions

Error Models

Tags

Versioning

Volume 3 Deliverables

✅ **Authentication APIs**
✅ **Tenant APIs**
✅ **Property APIs**
✅ **Asset APIs**
✅ **Applicant APIs**
✅ **Lease APIs**
✅ **Reservation APIs**
✅ **Utility APIs**
✅ **Payment APIs**
✅ **Document APIs**
✅ **Reporting APIs**
✅ **Mobile APIs**
✅ **Webhook Contracts**
✅ **Error Standards**
✅ **OpenAPI Governance**

Volume 4 - Terraform & Azure Blueprint

Unified Property Management SaaS Platform

Infrastructure as Code, Azure Landing Zone & Automated Tenant Provisioning

---

1. Purpose

This volume defines the infrastructure architecture and Infrastructure-as-Code (IaC) strategy for the platform.

Objectives:

- Fully automated infrastructure
- Zero manual Azure configuration
- Automated tenant provisioning
- Repeatable environments
- Enterprise-grade security

- White-label deployment support
- Hybrid tenancy support

This document serves as the implementation blueprint for:

- Terraform
- Azure
- GitHub Actions
- Container Apps
- Disaster Recovery
- Customer Onboarding Automation

---

2. Infrastructure Principles

IA-001 Infrastructure as Code

All infrastructure must be deployed via:

Terraform

No production resources created manually.

IA-002 Immutable Infrastructure

Infrastructure updates occur via:

Terraform Apply

Never modify resources directly.

IA-003 Environment Parity

All environments use identical modules.

Dev

QA

UAT

Production

Differences must exist only in configuration.

IA-004 Tenant Automation

Enterprise tenants must be provisioned automatically.

Target provisioning:

< 15 Minutes

---

3. Azure Landing Zone

Management Hierarchy

Management Group

├ Platform

├ Security

├ Shared Services

├ Non-Production

│ ├ Dev

│ ├ QA

│ └ UAT

└ Production

Subscription Strategy

Dev

PropertyPlatform-Dev

QA

PropertyPlatform-QA

UAT

PropertyPlatform-UAT

Production

PropertyPlatform-Prod

Enterprise Tenant Subscriptions

Optional:

PremierPM-Prod

TexasRV-Prod

StorageGroup-Prod

For large dedicated customers.

---

4. Terraform Repository Structure

terraform/

├ modules/

│ ├ networking

│ ├ frontdoor

│ ├ container_apps

│ ├ postgres

│ ├ redis

│ ├ keyvault

│ ├ storage

│ ├ monitoring

│ ├ dns

│ ├ tenant

│ └ white_label

├ environments/

│ ├ dev

│ ├ qa

│ ├ uat

│ └ prod

├ customer-templates/

├ scripts/

└ policies/

---

5. Core Terraform Modules

Module: Networking

Purpose:

VNET

Subnets

Private Endpoints

DNS Zones

Output:

vnet_id

private_subnet_ids

Module: Front Door

Purpose:

Global Routing

Load Balancing

SSL

WAF

Outputs:

frontdoor_hostname

waf_policy_id

Module: Container Apps

Purpose:

Host Microservices

Deploy:

Gateway

Auth

Property

Lease

Reservation

Utility

Payments

Reporting

Module: PostgreSQL

Purpose:

Primary Database

Read Replicas

Backups

Module: Redis

Purpose:

Caching

Session Storage

Event Processing

Module: Monitoring

Purpose:

Application Insights

Log Analytics

Alerts

---

6. Shared SaaS Environment Blueprint

Architecture

Azure Front Door

|

API Gateway

|

Shared Container Apps

|

PostgreSQL

|

Blob Storage

Supported Scale

1,000 Customers

250,000 Units

1,000,000 Residents

Isolation

tenant_id

Required on every business entity.

---

7. Enterprise Environment Blueprint

Enterprise plan customers receive:

Dedicated Database

Dedicated Storage

Dedicated Monitoring

Dedicated Domain

Architecture

Customer Domain

|

Azure Front Door

|

Dedicated API Gateway

|

Dedicated Service Group

|

Dedicated Database

---

8. White Label Infrastructure Blueprint

White Label Features

Custom Domain

Custom Branding

Custom Email Templates

Custom Support Information

Example

portal.premierpm.com

managed by

property-platform.com

Components

Brand Storage

DNS

Certificate

Email Templates

---

9. Azure Container Apps Blueprint

Container Apps Environment:

property-platform-prod

Services:

gateway

auth-service

tenant-service

property-service

lease-service

reservation-service

utility-service

payment-service

maintenance-service

document-service

reporting-service

erp-adapter

Autoscaling

Minimum:

minReplicas: 1

Maximum:

maxReplicas: 20

Production workloads:

maxReplicas: 50

---

10. Database Blueprint

PostgreSQL Flexible Server

Purpose:

Transactional Workloads

Separate Schemas

tenant

property

leasing

reservation

utility

payments

documents

reporting

High Availability

Configure:

Zone Redundant

Automatic Failover

---

11. Storage Blueprint

Azure Blob Storage Containers:

leases

documents

inspections

photos

contracts

exports

branding

Retention

7 Years

for legal documents.

---

12. Key Vault Blueprint

Store:

Database Passwords

OAuth Secrets

Stripe Keys

DocuSign Keys

JWT Signing Keys

Rules:

No secrets in code

No secrets in pipelines

---

13. Monitoring & Observability

Deploy:

Azure Monitor

Log Analytics

Application Insights

Metrics

Track:

CPU

Memory

Requests

Errors

Latency

Business Metrics

Track:

Reservations

Payments

Leases

Occupancy

Revenue

---

14. Customer Provisioning Automation

New Shared Customer

Workflow:

Subscription Purchased

↓

Tenant Created

↓

Database Records Created

↓

Branding Applied

↓

User Invited

↓

Active

Target time:

< 5 minutes

New Enterprise Customer

Workflow:

Subscription Purchased

↓

Terraform Pipeline

↓

Dedicated Infrastructure

↓

Domain Configuration

↓

Certificate Creation

↓

User Provisioning

↓

Active

Target:

10-15 minutes

---

15. Terraform Enterprise Customer Example

```
module "tenant" {
  source = "../modules/tenant"
  tenant_name = "Premier PM"
  tenant_type = "enterprise"
  custom_domain = "portal.premierpm.com"
  tier = "enterprise"
}
```

---

16. GitHub Actions Deployment Pipeline

Push

↓

Build

↓

Unit Tests

↓

Security Scan

↓

Terraform Plan

↓

Approval Gate

↓

Terraform Apply

↓

Deploy Containers

↓

Smoke Tests

↓

Success

---

17. DNS Architecture

Hosted in:

Azure DNS

Examples:

api.propertyplatform.com

tenant.propertyplatform.com

portal.premierpm.com

rv.texasretreats.com

---

18. Backup & Recovery

Database

Frequency:

Every 15 Minutes

Retention:

35 Days

Blob Storage

Soft Delete

Versioning

Geo-Redundancy

---

19. Disaster Recovery Blueprint

Primary:

South Central US

Secondary:

Central US

Recovery Objectives

Shared SaaS:

RPO: 15 Minutes

RTO: 4 Hours

Enterprise:

RPO: 5 Minutes

RTO: 1 Hour

---

20. Security Controls

Infrastructure Security:

Private Endpoints

Managed Identity

Key Vault

TLS 1.3

WAF

DDoS Protection

Network Security

Databases:

Private Only

Public database endpoints prohibited.

---

21. Cost Model

Shared Environment

Target:

100-500 Customers

Estimated Monthly Cost:

$250-$1,000

Enterprise Environment

Per customer:

$50-$300/month

depending on scale.

---

22. Environment Promotion Strategy

Developer Branch

|

Dev

|

QA

|

UAT

|

Production

Deployment approvals required before production.

---

23. Infrastructure Naming Standards

Example:

aca-auth-prod

aca-reservation-prod

pg-platform-prod

kv-platform-prod

Convention:

resource-service-environment

---

24. Terraform State Management

Use:

Azure Storage Backend

Example:

```
terraform {
 backend "azurerm" {
  resource_group_name  = "tfstate-rg"
  storage_account_name = "tfstateprod"
  container_name       = "state"
  key                  = "prod.tfstate"
 }
}
```

---

25. Volume 4 Deliverables

- ✅ Azure Landing Zone
- ✅ Subscription Strategy
- ✅ Shared SaaS Blueprint
- ✅ Enterprise Tenant Blueprint
- ✅ White Label Blueprint
- ✅ Container Apps Architecture
- ✅ PostgreSQL Architecture
- ✅ Storage Architecture
- ✅ Key Vault Architecture
- ✅ Monitoring Architecture
- ✅ Customer Provisioning Automation
- ✅ Terraform Module Catalog
- ✅ Deployment Pipelines
- ✅ Backup & DR Strategy
- ✅ Cost Model

Volume 5 - GitHub Engineering Blueprint

Unified Property Management SaaS Platform

Source Control, Engineering Standards, DevSecOps & AI-Assisted Development Operating Model

---

1. Purpose

This volume defines the engineering operating model for the platform.

It establishes:

- GitHub organization structure
- Monorepo strategy
- Branching standards
- Coding standards
- Pull request standards
- GitHub Actions standards
- DevSecOps controls
- AI-assisted development standards
- Release management
- Documentation requirements

This document becomes the authoritative guide for:

Developers

Architects

DevOps Engineers

QA Engineers

Platform Engineers

AI Coding Agents

---

2. Engineering Principles

ENG-001

Platform First

Avoid custom implementations when the capability can be delivered through:

ERPNext

Azure

GitHub

Terraform

ENG-002

Everything as Code

All artifacts shall exist in Git.

Infrastructure

Configuration

Policies

APIs

Schemas

Documentation

ENG-003

AI-Native Development

Every repository must be structured so that:

GitHub Copilot

Claude Code

OpenAI

Cursor

can understand and contribute safely.

ENG-004

Documentation Required

Code without documentation is considered incomplete.

---

3. GitHub Organization Strategy

GitHub Organization

property-platform

Teams

Architecture

Platform

Backend

Frontend

Mobile

DevOps

Security

Data

Operations

Permission Model

Role	Permission
Architecture	Admin

Role	Permission
Platform Lead	Maintain
Developers	Write
Contractors	Limited Write
AI Generated PRs	Review Required

---

4. Repository Strategy

Monorepo Architecture

Approved model:

property-platform

Repository Structure

property-platform/

├ docs/

├ architecture/

├ terraform/

├ deployments/

├ shared/

├ frontend/

├ mobile/

├ services/

│ ├ gateway

│ ├ identity

│ ├ tenant

│ ├ property

│ ├ lease

│ ├ reservation

│ ├ utility

│ ├ payment

│ ├ maintenance

│ ├ document

│ ├ reporting

│ └ erp-adapter

├ tests/

└ scripts/

---

5. Architecture Folder Standards

architecture/

├ volume-1

├ volume-2

├ volume-3

├ volume-4

├ volume-5

├ volume-6

├ adr

├ diagrams

├ erd

└ sequence

Required Formats

Markdown

Mermaid

PlantUML

PDF

---

6. Branching Strategy

Long-Lived Branches

main

develop

Short-Lived Branches

feature/

bugfix/

hotfix/

release/

Examples

feature/reservation-service

feature/utility-billing

bugfix/payment-retry

---

7. Pull Request Standards

Required Checks

Before merge:

Build Success

Tests Pass

CodeQL Pass

Dependency Scan Pass

Lint Pass

Architecture Review

PR Template

Required Sections:

Purpose

Changes

Testing

Security Impact

Database Impact

Rollback Strategy

---

8. Semantic Versioning

Standard:

MAJOR.MINOR.PATCH

Examples:

1.0.0

1.1.0

1.1.1

2.0.0

Service Versioning

Each service versioned independently.

Example:

reservation-service

v1.2.1

---

9. API Standards

All APIs must follow:

OpenAPI 3.1

Repository Structure

service/

├ src

├ tests

├ docs

└ openapi

API Definition

openapi.yaml

Required.

---

10. Backend Standards

Framework

Approved:

Java 21

Spring Boot

Package Structure

api

application

domain

infrastructure

integration

DDD Layers

Controller

Application

Domain

Repository

---

11. Frontend Standards

Framework

Next.js

TypeScript

Tailwind

ShadCN

Folder Structure

src/

├ app

├ components

├ features

├ hooks

├ services

├ types

└ utils

State Management

Approved:

Zustand

---

12. Mobile Standards

Framework

React Native

Future evaluation:

Flutter

Supported Platforms

iOS

Android

---

13. Database Standards

PostgreSQL

Required:

UUID Keys

Indexes

Constraints

Migrations

Migration Tool

Approved:

Flyway

Naming Rules

Tables:

snake_case

Example:

meter_reading

---

14. Terraform Standards

Directory Structure

terraform

├ modules

├ environments

├ policies

└ scripts

Module Rules

Modules must:

Be Reusable

Be Versioned

Be Documented

---

15. GitHub Actions Strategy

Required Workflows

Build

Unit Test

Integration Test

Lint

Security Scan

Terraform Plan

Deploy

Workflow Structure

.github/

workflows/

├ build.yml

├ test.yml

├ security.yml

├ deploy.yml

└ terraform.yml

---

16. Security Engineering Standards

Mandatory Scans

CodeQL

Required.

Dependabot

Required.

Secret Scanning

Required.

Container Scanning

Required.

---

17. DevSecOps Gates

Merge blocked if:

Critical Vulnerabilities

Failed Tests

Failed Build

Failed Security Scan

exist.

---

18. Testing Strategy

Test Pyramid

Unit Tests

Integration Tests

Contract Tests

System Tests

Coverage Targets

Area	Coverage
Domain Logic	90%
APIs	80%
Utilities	90%

Area	Coverage
Critical Workflows	95%

---

19. Contract Testing

All services publish:

OpenAPI Contract

Use:

Consumer Contract Tests

between services.

---

20. Documentation Standards

Each service must contain:

README

Architecture

API Spec

Deployment Guide

Runbook

Required Structure

/docs

README.md

architecture.md

deployment.md

runbook.md

---

21. Engineering Runbooks

Each service requires:

Startup

Shutdown

Recovery

Troubleshooting

Escalation

documentation.

---

22. AI-Assisted Development Standards

Approved Tools

GitHub Copilot

Claude Code

OpenAI

Cursor

AI Usage Rules

AI may:

Generate Code

Generate Tests

Generate Documentation

Suggest Refactoring

AI may not:

Approve PRs

Bypass Reviews

Deploy Production

---

23. Copilot Context Architecture

Repository must expose:

Architecture

ADRs

OpenAPI Specs

ERDs

Terraform Modules

for Copilot grounding.

Recommended Folder

.ai-context/

Contents:

ADR Catalog

ERD Package

Architecture Summaries

Business Glossary

---

24. AI Prompt Standards

Example:

Generate Reservation Service endpoint

Use OpenAPI v3.1

Follow platform standards

Use tenant isolation

Return DTOs

Add tests

Prohibited Prompts

Generate entire platform

without context.

---

25. Release Management

Release Cadence

Shared SaaS:

Biweekly

Enterprise:

Monthly

Emergency Fixes:

Hotfix Branch

---

26. Observability Standards

Every service must expose:

Metrics

Health Checks

Logs

Tracing

Endpoints:

/actuator/health

/actuator/metrics

---

27. Engineering KPIs

Track:

Deployment Success Rate

Lead Time

Change Failure Rate

Mean Time To Recovery

Target Values

KPI	Target
Deployment Success	>95%
MTTR	<1 Hour
Build Success	>98%
Critical Defects	<5/Release

---

28. Architecture Governance

Architecture Review Board required for:

Schema Changes

Major Integrations

New Services

Security Decisions

ADR Repository

architecture/adr

becomes the official system of record.

---

29. Developer Onboarding

New developer setup target:

< 60 Minutes

Must provide:

Docker Compose

Local PostgreSQL

Local Redis

Mock Services

---

30. Engineering Deliverables

Every feature must include:

- ✅ Code
- ✅ Tests
- ✅ OpenAPI Updates
- ✅ Documentation Updates
- ✅ Terraform Updates (if needed)
- ✅ Monitoring Updates
- ✅ Security Review

---

31. Volume 5 Deliverables

- ✅ GitHub Organization Strategy
- ✅ Monorepo Blueprint
- ✅ Branching Standards
- ✅ Pull Request Standards
- ✅ Semantic Versioning

✅ Coding Standards
✅ Terraform Standards
✅ GitHub Actions Standards
✅ DevSecOps Controls
✅ Testing Strategy
✅ Documentation Standards
✅ Runbook Standards
✅ AI-Assisted Development Standards
✅ Release Management
✅ Engineering Governance

Volume 6 - MVP Delivery Plan

Unified Property Management SaaS Platform

Product Delivery, Roadmap, Team Structure, Budget & Go-Live Strategy

---

1. Purpose

Volume 6 converts Volumes 1–5 into an executable delivery plan.

It defines:

- MVP scope
- Product roadmap
- Epics
- User stories
- Acceptance criteria
- Team structure
- Budget estimates
- Risk management
- Delivery strategy
- Go-live plan

The objective is to reach market with a commercially viable product in the shortest practical timeframe while preserving long-term architectural integrity.

---

2. MVP Philosophy

Core Principle

Do not build Rent Manager, AppFolio, Campspot, and Yardi on Day 1.

Build:

The smallest platform that:

Acquires customers

Collects revenue

Validates product-market fit

MVP Target Customers

Segment A

Small landlords

1-50 Units

Segment B

Independent PM firms

10-500 Units

Segment C

Small RV operators

20-250 Sites

---

3. MVP Scope

Included

Identity

- ✅ Microsoft Login
- ✅ Google Login
- ✅ Email Login

Property Management

- ✅ Properties
- ✅ Buildings
- ✅ Units
- ✅ RV Sites
- ✅ Cabins

Leasing

- ✅ Applicants
- ✅ Residents
- ✅ Leases
- ✅ Renewals

Reservations

- ✅ Availability Search
- ✅ Booking
- ✅ Check-In
- ✅ Check-Out

Payments

- [x] Stripe
- [x] ACH
- [x] Credit Cards

Documents

- [x] Lease Storage
- [x] E-Sign Workflow

Reporting

- [x] Occupancy
- [x] Revenue
- [x] Delinquency

Mobile APIs

- [x] Resident APIs
- [x] Guest APIs
- [x] Manager APIs

---

4. Post-MVP Features

Phase 2

Maintenance

Vendor Portal

SMS Platform

Advanced Reporting

Power BI Integration

Phase 3

Utility Billing

RUBS

Dynamic Pricing

Seasonal Rates

Phase 4

Advanced White Label

Marketplace

Owner Portal

Investor Reporting

Phase 5

AI Agents

Collections Agent

Leasing Agent

Revenue Agent

---

5. MVP Epics

Epic 1

Identity & Tenant Management

Goal:

Secure User Access

Deliverables:

Authentication

RBAC

Tenant Provisioning

Epic 2

Property Management

Goal:

Create Asset Hierarchy

Deliverables:

Properties

Buildings

Units

Sites

Epic 3

Leasing

Goal:

Manage Residents

Deliverables:

Applicants

Leases

Renewals

Epic 4

Reservations

Goal:

Manage Transient Occupancy

Deliverables:

Availability

Bookings

Check-In

Check-Out

Epic 5

Payments

Goal:

Collect Revenue

Deliverables:

Stripe

Invoices

Payments

Epic 6

Documents

Goal:

Store Leases

Support Signatures

Epic 7

Reporting

Goal:

Dashboard Visibility

---

6. Detailed User Stories

Story

Create Property

As a

Property Manager

I want

to create a property

So that

I can manage assets.

Acceptance Criteria

Property Saved

Property Visible

Audit Logged

Tenant Isolated

Story

Create Lease

As a

Property Manager

I want

to create a lease

So that

a resident can move in.

Acceptance Criteria

Lease Created

Resident Linked

Document Generated

Audit Recorded

Story

Reserve RV Site

As a

Guest

I want

to reserve a site online

So that

I can stay at the park.

Acceptance Criteria

Availability Verified

Rate Applied

Reservation Created

Payment Processed

---

7. MVP Team Structure

Lean Team

Product

Product Owner

Architecture

Principal Architect

Engineering

2 Backend Developers

1 Frontend Developer

1 Full Stack Developer

DevOps

1 DevOps Engineer

QA

1 QA Engineer

Total:

7 People

---

8. AI-Augmented Team Option

Given modern AI tooling:

GitHub Copilot

Claude

OpenAI

Cursor

The team could be:

Product Owner

Principal Architect

2 Full Stack Engineers

1 DevOps Engineer

Total:

4 People

---

9. Development Methodology

Scrum

Sprint Length:

2 Weeks

Release Cadence

Development:

Biweekly

Production:

Monthly

---

10. Sprint Plan

Sprint 1-2

Platform Foundation

Deliver:

GitHub

Terraform

Azure

Container Apps

Identity

Sprint 3-4

Property Domain

Deliver:

Properties

Assets

Tenants

Sprint 5-6

Leasing Domain

Deliver:

Residents

Applicants

Leases

Sprint 7-8

Reservation Domain

Deliver:

Availability

Bookings

Sprint 9-10

Payments

Deliver:

Invoices

Stripe

Payments

Sprint 11-12

Reporting

Deliver:

Dashboards

Metrics

---

11. MVP Timeline

Conservative

9 Months

AI Accelerated

4-6 Months

Recommended Target

6 Months

---

12. Initial Budget

Lean Team

6 Months

Estimated:

$250,000 - $500,000

AI-Augmented Team

6 Months

Estimated:

$100,000 - $250,000

---

13. Monthly Operating Cost

Shared SaaS

Estimated:

$250-$1,000/month

Enterprise Customer

Additional:

$50-$300/month

---

14. Revenue Model

Shared SaaS

Example:

$49/month

$99/month

$199/month

Enterprise

Example:

$499/month

$999/month

$2,499/month

White Label

Example:

$2,500-$10,000/year

---

15. Risk Register

Risk

Scope Creep

Mitigation:

Strict MVP

Risk

ERPNext Customization Complexity

Mitigation:

Adapter Layer

Risk

Payment Integration Delays

Mitigation:

Stripe First

Risk

Multi-Tenant Security

Mitigation:

Tenant Isolation Testing

Risk

AI Hallucinations

Mitigation:

RAG

Approval Workflows

---

16. Go-Live Strategy

Alpha

Internal Users

10 Properties

Beta

Pilot Customers

5 Property Managers

2 RV Parks

General Availability

Launch Target:

50 Customers

2,500 Units/Sites

---

17. Customer Migration Strategy

Phase 1

Manual Imports

CSV

Phase 2

Automated Migrations

Rent Manager

Innago

Buildium

Phase 3

Advanced Migrations

Yardi

AppFolio

Campspot

---

18. Go-Live Success Metrics

Technical

99.9% Availability

<300ms API Latency

Business

25 Paying Customers

1,000 Units

90% Customer Retention

within the first year.

---

19. Post Go-Live Support

Tier 1

Customer Success

Tier 2

Operations

Tier 3

Engineering

Support Hours:

Business Hours Initially

24x7 For Enterprise Customers

---

20. Strategic Recommendation

Phase 1

Focus on:

Property Management

Leasing

Reservations

Payments

Documents

Do **not** build:

Advanced Utility Billing

Marketplace

AI Agents

Complex Owner Accounting

yet.

Phase 2

Add:

Utility Billing

Advanced Reporting

White Label Features

Phase 3

Add:

AI Leasing

AI Reservations

AI Reporting

AI Collections

Volume 6 Deliverables

- ✅ Product Backlog
- ✅ Epics
- ✅ User Stories
- ✅ Acceptance Criteria
- ✅ Sprint Plan
- ✅ Team Plan
- ✅ Budget Model
- ✅ Revenue Model
- ✅ Risk Register
- ✅ Go-Live Plan
- ✅ Customer Migration Plan
- ✅ Success Metrics

Architecture Package Complete

You now have the complete blueprint set:

Volume 1 – Architecture Master Document
Volume 2 – Data Architecture & ERD Package
Volume 3 – OpenAPI Specification
Volume 4 – Terraform & Azure Blueprint
Volume 5 – GitHub Engineering Blueprint
Volume 6 – MVP Delivery Plan

Final Architect Recommendation